음성 AI 전문가 과정 · Day 5 / 40

Day 05

딥러닝 오디오 분류
Speech Commands 키워드 인식

데이터셋 → Log-Mel → CNN → softmax · loss → 학습 루프 → 혼동 행렬 → SpecAugment → predict_keyword()
4 시간 · Colab T4 GPU 권장 · 실습 노트북 Part 1 ~ 6

```
epoch  1 | train acc 52% | val acc 71%
epoch  6 | train acc 89% | val acc 88%
epoch 12 | train acc 95% | val acc 91%

>>> predict_keyword("my_yes.wav", model)
[('yes', 0.94), ('left', 0.03), ...]
```

출력 예시 (값은 실행마다 다름)

0.0 · 오늘 답할 질문

1 초 음성을 듣고 단어를 맞히는 모델은 어떻게 만들까

1	데이터를 학습 · 테스트로 나눌 때 왜 " 화자 " 를 기준으로 할까 ?	Part 1
2	1 초 음성은 CNN 에게 어떤 모양의 입력이 될까 ?	Part 2
3	CNN 의 출력 크기와 파라미터 수는 어떻게 계산할까 ? softmax 와 loss 는 ?	Part 3
4	학습 한 걸음은 어떤 네 줄로 이루어질까 ? 과적합은 어떻게 알아볼까 ?	Part 4 · 5
5	내 음성 파일을 넣어 맞히려면 무엇을 학습 때와 똑같이 해야 할까 ?	Part 6

오늘 처음으로 " 학습하는 모델 " 을 만든다 . D6 Whisper 를 이해하는 바탕이다 .

1초 음성 → Log-Mel → CNN → "yes"

D5 · 키워드 인식(KWS) 모델의 흐름

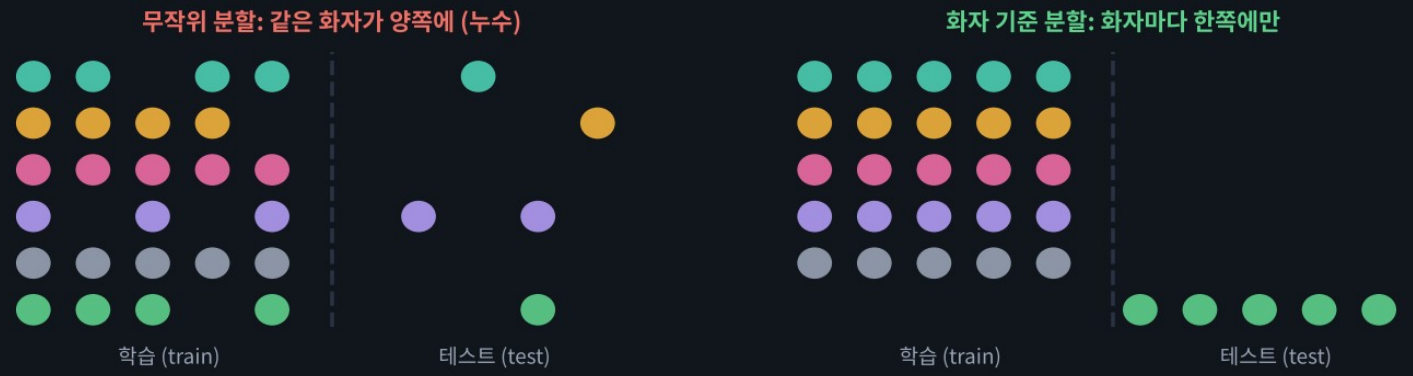


학습: 정답과 비교해 cross-entropy 손실 → backward → 파라미터 업데이트 (Part 4)

교시	내용	실습
1 교시	Speech Commands, 화자 분할 , 1 초 맞추기 , Log-Mel, Dataset · DataLoader	Part 0 ~ 2 · 빈칸 1 ~ 3 · 워크시트 1 교시
2 교시	합성곱 · 출력 크기 · 파라미터 , KWSNet, softmax · cross-entropy	Part 3 · 빈칸 4 ~ 5 · 워크시트 2 교시
3 교시	경사 하강법 , 학습 루프 , 학습 곡선 , 혼동 행렬	Part 4 ~ 5 · 빈칸 6 ~ 7 · 워크시트 3 교시
4 교시	SpecAugment, predict_keyword(), 모델 저장	Part 6 ~ 7 · 빈칸 8 ~ 9 · 워크시트 4 교시 · 노션

▶ **애니메이션** D5-1 오늘의 흐름 (Part 1~6)

짧은 단어 하나를 알아듣는 데이터셋



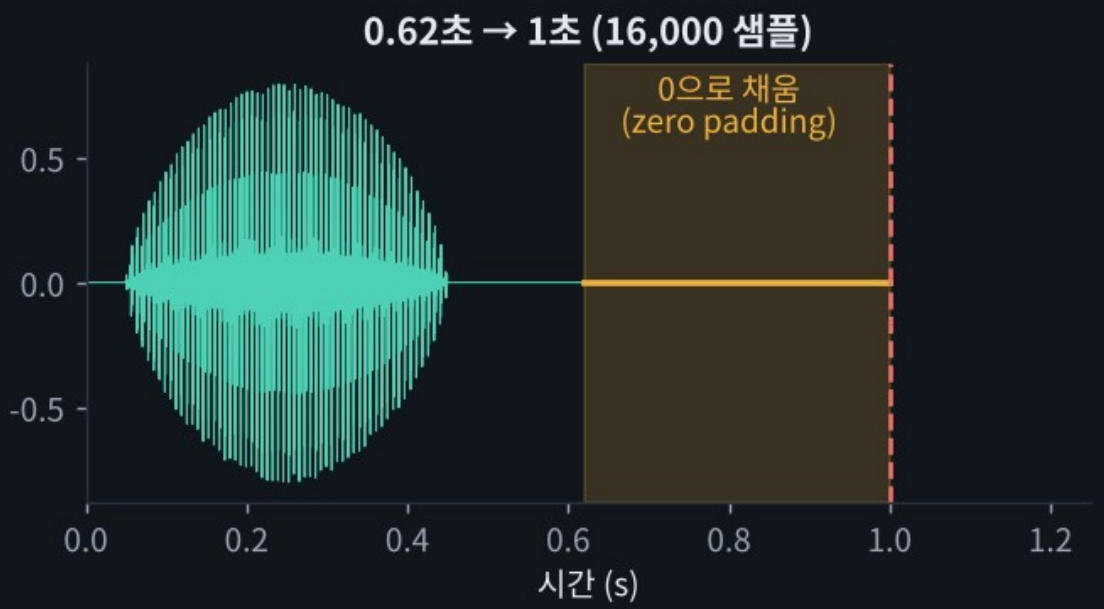
항목	값
단어	35 개 (오늘은 10 개)
클립	약 10 만 5 천 개
길이	1 초 이하 · 16 kHz
화자	2,600 명 이상
분할	학습 80 · 검증 10 · 테스트 10

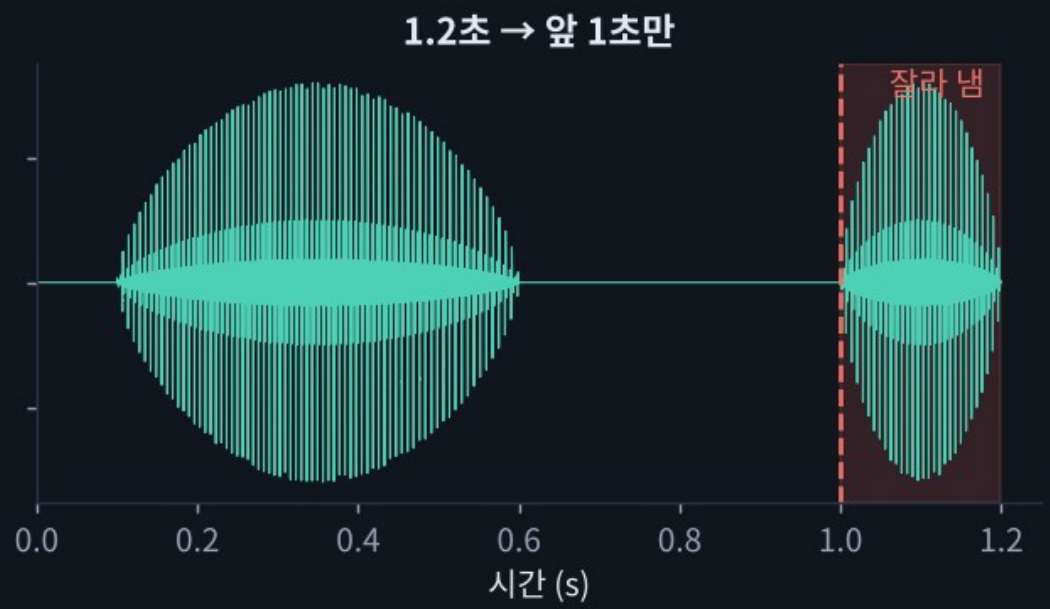
```
train_meta = torchaudio.datasets.SPEECHCOMMANDS("data", download=True, subset="training")
rel, sr, word, speaker, n = train_meta.get_metadata(i)      #  오디오는 soundfile 로 직접 읽기
```

split(speaker) = hash(speaker) mod 100 → 0~9 검증 · 10~19 테스트 · 나머지 학습

▶ **애니메이션** D5-2 데이터셋과 분할 (누수 비교)

모든 클립을 16,000 샘플로 , 그리고 (1, 64, 101) 로



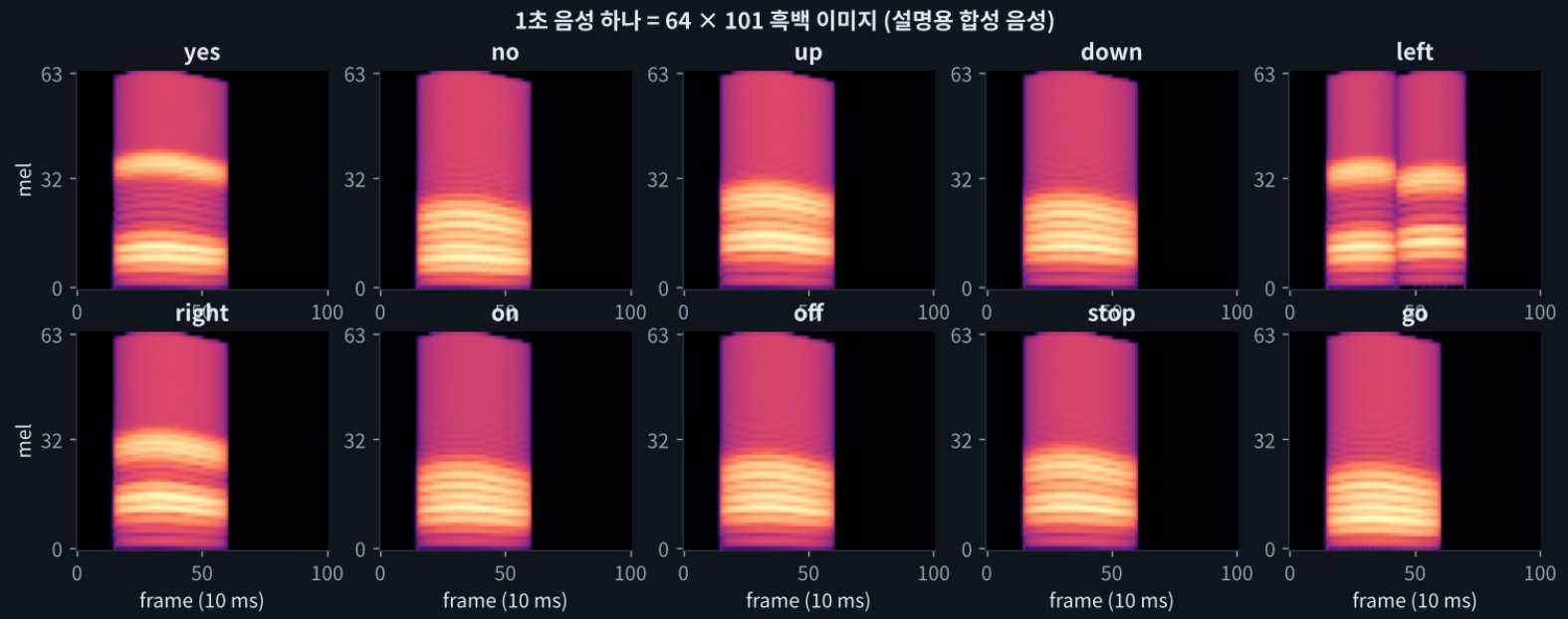


🧩 **빈칸 실습 1** `np.pad(y, (0, ____ - len(y)))` 정답 : n

🧩 **빈칸 실습 2** `n_frames = 1 + SR // ____` 정답 : HOP (160)

단계	설정	모양
파형	1 초 · 16 kHz	`(1, 16000)`
MelSpectrogram	n_fft 400 · hop 160 · n_mels 64	`(1, 64, 101)`
AmplitudeToDB	10·log10, top_db 80	`(1, 64, 101)`

1 초 음성 하나 = 64 × 101 흑백 이미지




```
MEL = T.MelSpectrogram(
  sample_rate=16000,
  n_fft=400,
  hop_length=160,
  n_mels=64)
TO_DB = T.AmplitudeToDB(
  top_db=80)

x̂ = (x − μ) / σ
μ, σ =  학습 세트 기준
```

⚠ 정규화 원칙

검증 · 테스트 · 예측에도 학습 세트의 μ, σ 를 그대로

▶ **애니메이션** D5-3 1초 음성 → Log-Mel (n_mels · hop)

" 몇 개인가 " 와 "i 번째는 무엇인가 " 만 알려 주면 된다

```
class FeatDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        return ____, self.y[i]

train_loader = DataLoader(ds, batch_size=128,
                          shuffle=True)
```

🧩 빈칸 실습 3

정답 : self.X[i]
DataLoader 가 128 번 불러 쌓아 한 배치로

텐서	모양
X_train	(N, 1, 64, 101)
한 배치 xb	(128, 1, 64, 101)
정답 yb	(128,)

학습 세트 크기 : 단어당 1,500 개 × 10 = 15,000 개 (GPU) · CPU 에서는 자동 축소

필터와 겹친 칸끼리 곱해서 더하기

입력 (Log-Mel 일부)

2	3	1	0	2	3	2	4	2
3	4	5	2	0	4	3	5	2
2	0	2	3	4	5	1	3	4
1	2	5	3	3	4	3	5	4
0	0	3	4	0	4	4	4	5
1	3	4	2	1	2	0	1	5
4	5	5	5	1	2	3	1	5

필터 3 × 3 (학습됨)

1	0	-1
1	0	-1
1	0	-1

겹친 칸끼리 곱해서 모두 더하기

$= \Sigma\, w \cdot x + b = 3 + b$

출력 크기 $H_{out} = \lfloor (H + 2P - K)/S \rfloor + 1$

K = 3, P = 1, S = 1 → 크기 유지

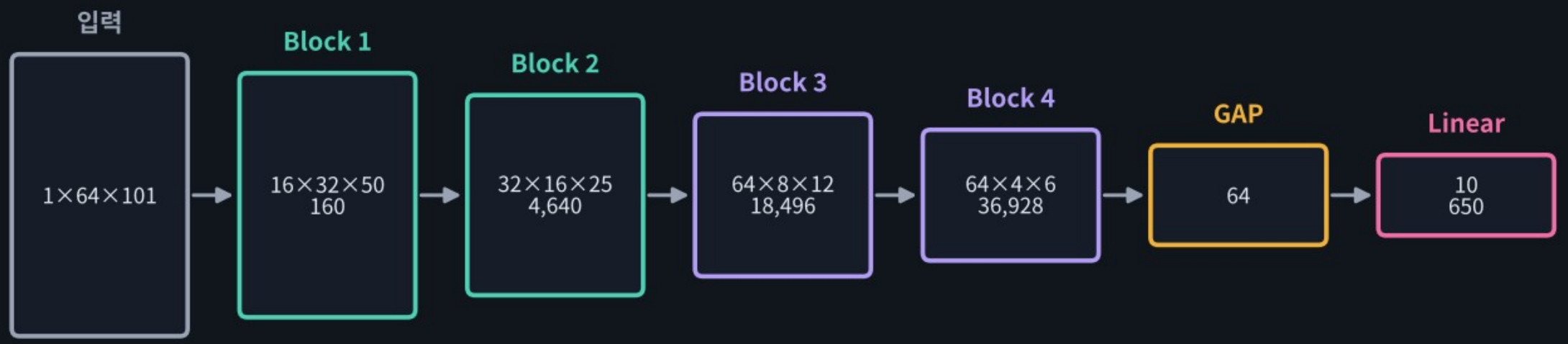
파라미터 $C_{in} \cdot C_{out} \cdot K^2 + C_{out}$

1 → 16채널: $1 \cdot 16 \cdot 9 + 16 = 160$

```
H_out = ⌊(H + 2P − K) / S⌋ + 1      (64 + 2 − 3)/1 + 1 = 64
params = C_in·C_out·K² + C_out      16·32·9 + 32 = 4,640
```

▶ **애니메이션** D5-4 합성곱 (K · P · S)

Conv 블록 4 개 + GAP + Linear: 약 6 만 개 파라미터



Block = Conv 3×3 (padding 1) → BatchNorm → ReLU → MaxPool 2 · 숫자 아래 = conv 파라미터 수 · 전체 약 61,000개

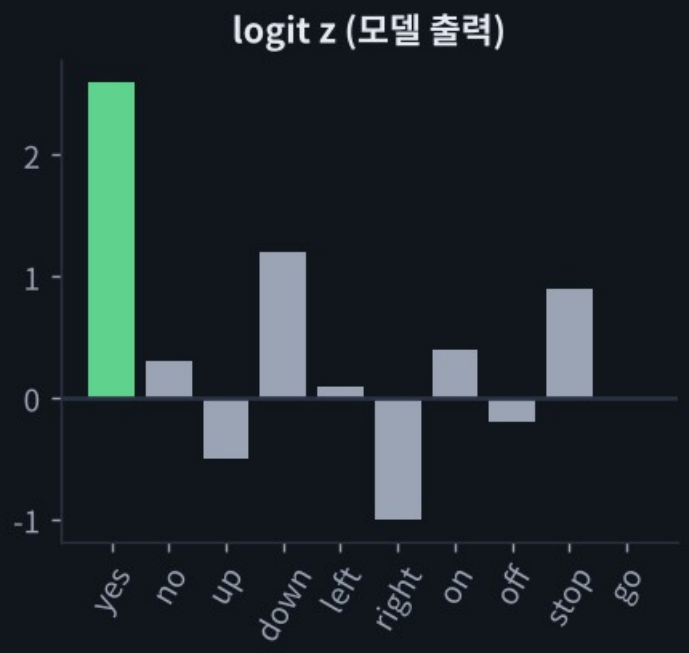
층	출력 모양	파라미터
Block 1 (1→16)	`(16, 32, 50)`	160 + BN 32
Block 2 (16→32)	`(32, 16, 25)`	4,640 + 64
Block 3 (32→64)	`(64, 8, 12)`	18,496 + 128
Block 4 (64→64)	`(64, 4, 6)`	36,928 + 128
GAP → Linear	`(64,)` → `(10,)`	650

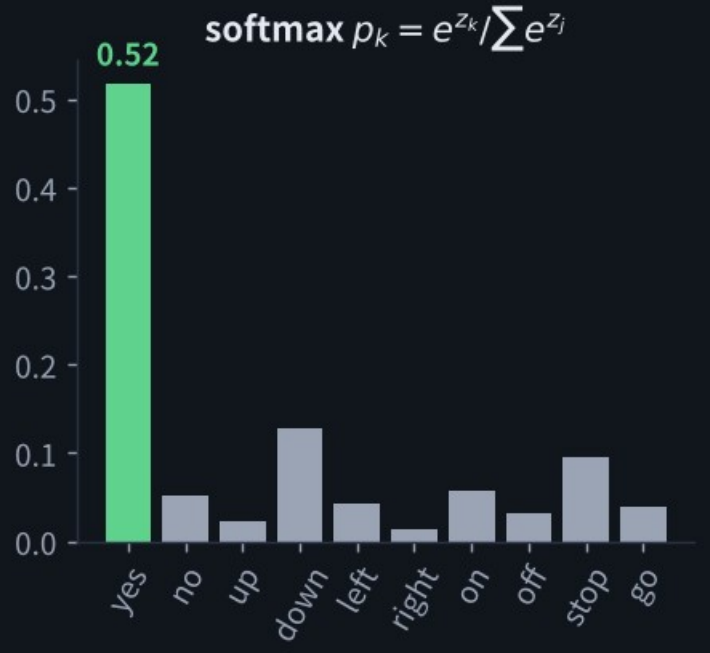
🧩 **빈칸 4** `Linear(64, ____)` 정답 : n_classes

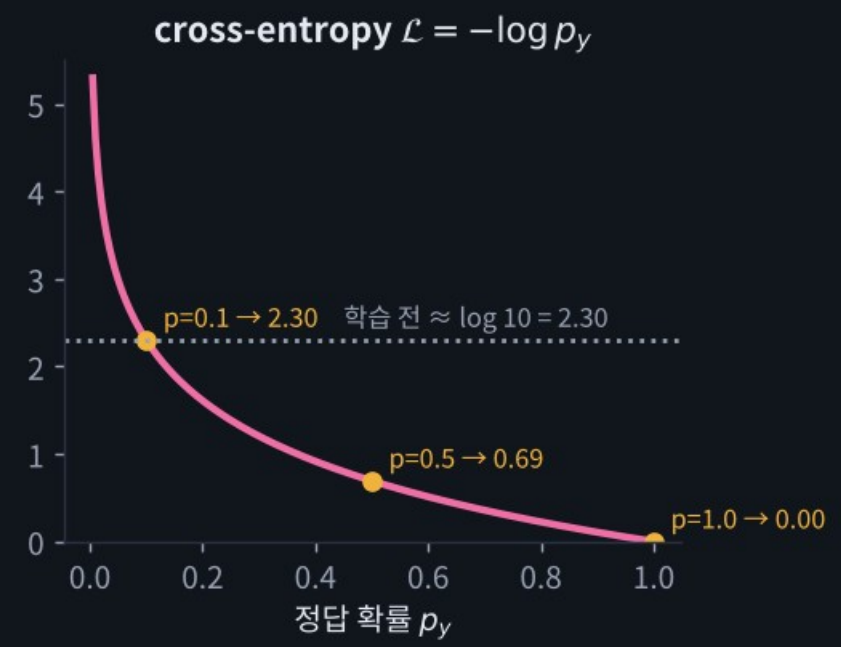
▶ **애니메이션** D5-5 모양 추적 · 파라미터

합계 61,226 개 · float32 약 240 KB

점수 → 확률 → 손실 – log p_y





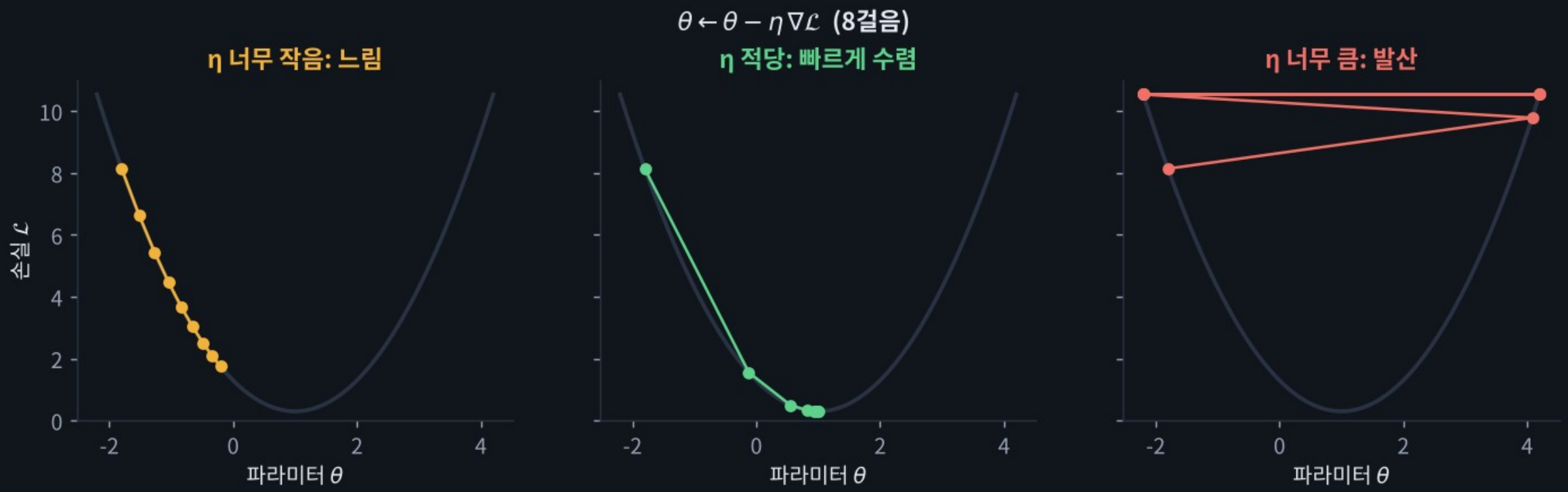



```
p_k = e^z_k / Σ e^z_j        L = -log p_y
∂L/∂z = p - onehot        ( 정답 칸은 올리고 ,  나머지는 내림 )
```

🧩 **빈칸 5** `.sum(dim=____)` 정답 : 1

▶ **애니메이션** D5-6 softmax · loss

θ ← θ − η∇L: 기울기 반대로 한 걸음



용어	뜻
미니배치	128 개씩 손실 계산 → 한 걸음
epoch	학습 데이터를 한 바퀴
Adam	걸음 크기 자동 조절 , η = 1e-3

▶ **애니메이션** D5-7 경사 하강법 · 학습 곡선

forward → loss → backward → step

```
for xb, yb in loader:
    xb, yb = xb.to(DEVICE), yb.to(DEVICE)
    logits = model(xb)                  # 1. forward
    loss = F.cross_entropy(logits, yb)  # 2. loss
    opt.zero_grad()
    loss.____()                        # 3. backward
    opt.____()                          # 4. update
```

🧩 빈칸 실습 6

정답 : backward, step

model.train() / model.eval()

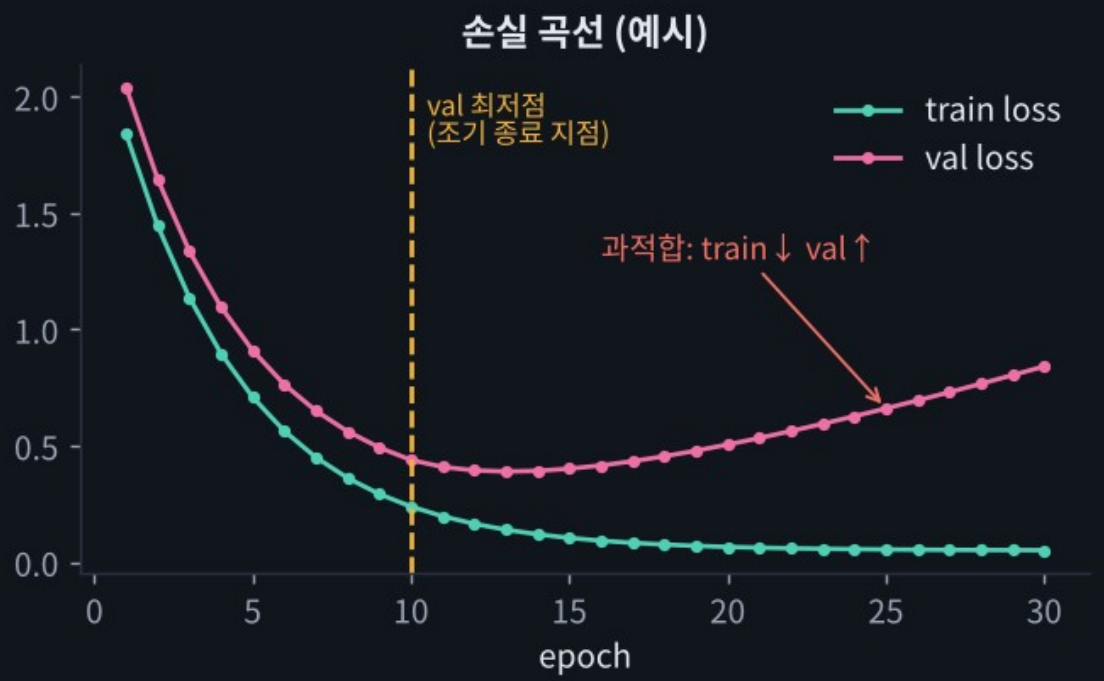
Dropout · BatchNorm 동작이 다름

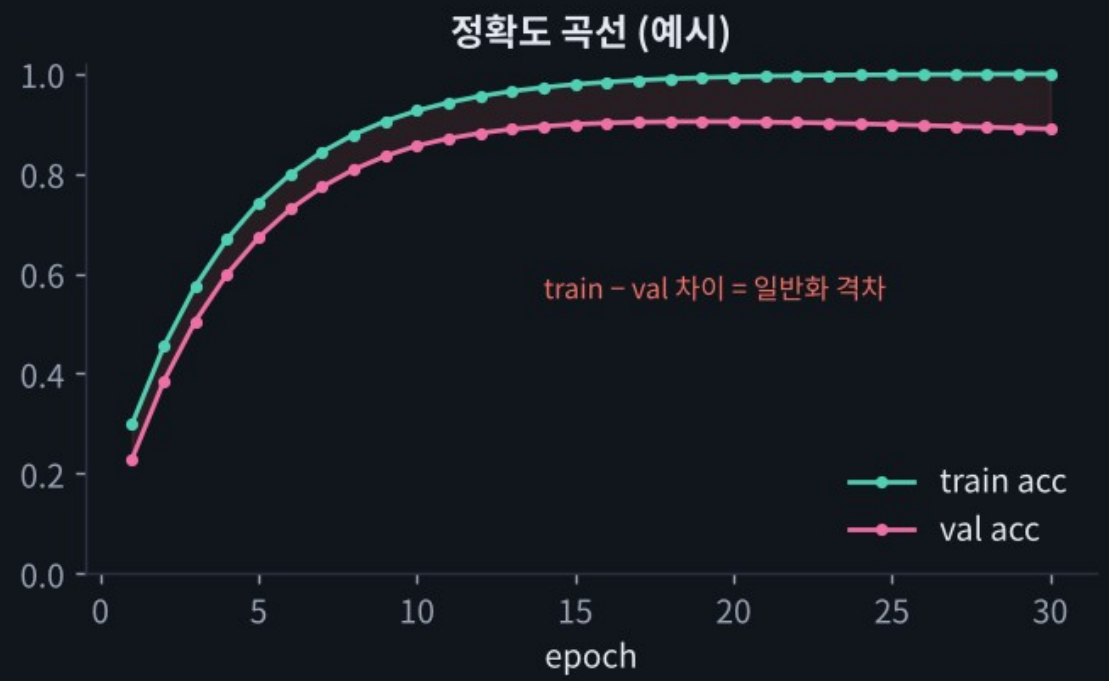
@torch.no_grad()

평가 중 기울기 계산 끔 → 빠르고 메모리 절약

학습 전 val loss ≈ 2.30 (= log 10) 에서 시작해 내려가는지 먼저 확인한다

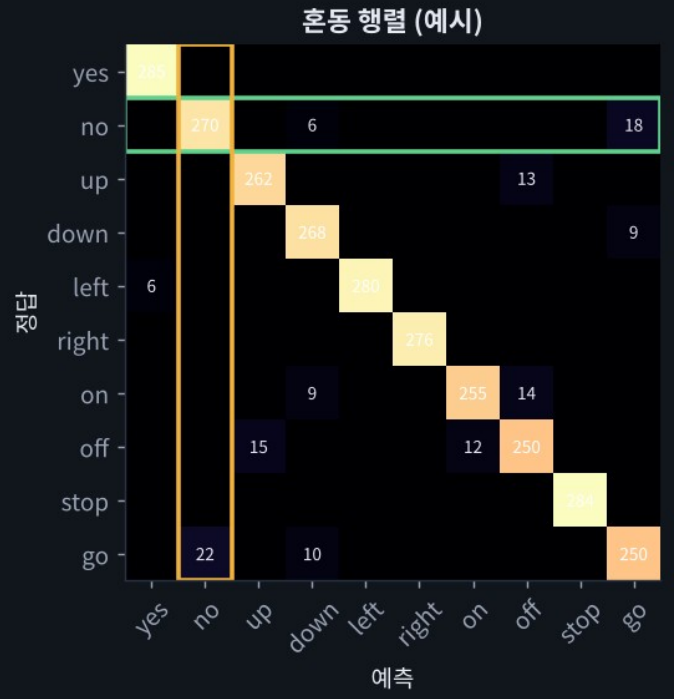
train 과 val 의 차이가 과적합을 알려 준다





모양	뜻	대응
train · val 같이 내려감	잘 배우는 중	epoch 더
train↓ · val↑	과적합	증강 · Dropout · 조기 종료
둘 다 높게 멈춤 · NaN	과소적합 · η 과다	모델 키우기 · η 조정

행 = 정답, 열 = 예측 : 무엇을 무엇으로 헷갈렸나



"no" 클래스 예시

재현율 recall = 대각 / 행 합 = 270/294 = 91.8%

진짜 "no" 중에서 맞힌 비율 (초록 행)

정밀도 precision = 대각 / 열 합 = 270/292 = 92.5%

"no"라고 예측한 것 중 진짜 비율 (주황 열)

F1 = 2PR/(P+R) = 92.2%

자주 헷갈리는 쌍: no ↔ go, on ↔ off, up ↔ off

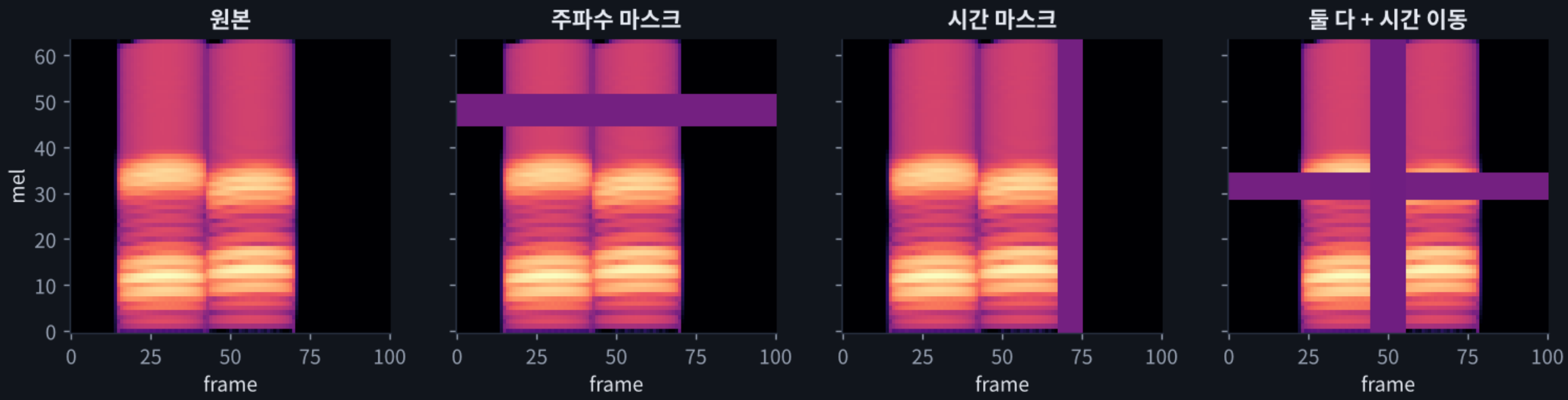
→ 모음이 비슷하고 자음 차이가 짧은 단어들

```
recall_k =
  C[k,k] / 행 합
precision_k =
  C[k,k] / 열 합
F1 = 2PR/(P+R)
acc = 대각 / 전체
```

🧩 **빈칸 7** `dim=____` 정답 : 1

▶ **애니메이션** D5-8 혼동 행렬

가려도 알아듣게 : 주파수 · 시간 마스크




```
SPEC_AUG = nn.Sequential(
    T.FrequencyMasking(freq_mask_param=8),
    T.TimeMasking(time_mask_param=12))
hist_aug = fit(model_aug, augment=____)
```

🧩 **빈칸 8** augment=____ 정답 : augment

▶ **애니메이션** D5-9 SpecAugment (🎲)

predict_keyword(): 아무 파일이나 넣으면 top-3

1 읽기
16 kHz mono
›
2 center_1s
무음 자르고 1초
›
3 Log-Mel
1 × 64 × 101
›
4 정규화
학습 세트 μ, σ
›
5 모델
softmax
›
6 top-3
(단어 , 확률)

```
y, _ = librosa.load(path, sr=SR, mono=True)
x = (wav2feat(center_1s(y)) - ____) / SD
p = F.softmax(model(x.unsqueeze(0)), dim=1)[0]
top = p.topk(3)

torch.save({"state_dict": ..., "keywords": ...,
            "mu": MU, "sd": SD}, "kws_cnn.pt")
```

🧩 빈칸 실습 9

정답 : MU (학습 세트 평균)

gTTS 로 시험

TTS 목소리는 학습 분포와 달라 일부 틀릴 수 있음

학습 때와 똑같은 전처리 = 같은 길이 · 같은 특징 설정 · 같은 μ, σ

오늘의 원리를 열두 줄로

원리	수식 · 코드	확인
화자 분할	`hash(speaker) mod 100`	학습 ∩ 테스트 화자 = 0
1초 맞추기	`np.pad(y, (0, n − len(y)))`	빈칸 1
프레임 수	`1 + ⌊16000/160⌋ = 101`	빈칸 2
정규화	`(x − μ)/σ, 학습 세트 기준`	빈칸 9
합성곱	`y = b + Σ w·x`	D5-4
출력 크기	`⌊(H + 2P − K)/S⌋ + 1`	수식 검증 1
파라미터	`C_in·C_out·K² + C_out`	61,226 개
softmax	`p_k = e^z_k / Σ e^z_j`	빈칸 5
cross-entropy	`L = −log p_y, ∂L/∂z = p − onehot`	autograd 비교
경사 하강	`θ ← θ − η∇L`	빈칸 6
재현율 · 정밀도	`C[k,k]/행 합 · C[k,k]/열 합`	수식 검증 2
SpecAugment	`FrequencyMasking · TimeMasking`	빈칸 8

오늘 헷갈리기 쉬운 것

헷갈리는 점	설명
torchaudio.load 오류	torchaudio 2.9+ 는 I/O 를 TorchCodec 으로 옮기는 중 . 오늘은 soundfile 로 읽음
Colab 에서 torchaudio 를 pip 로 다시 설치	torch 와 버전이 어긋나 import 오류 . 기본 설치 그대로 사용
dim=0 vs dim=1	(B, 10) 에서 클래스 방향은 dim=1. softmax 합 · argmax 모두
val 과 test 의 차이	val 은 설정 고르기용 , test 는 마지막 성적표 . test 로 고르면 안 됨
정확도가 실행마다 다름	무작위 초기화 · 섞기 · 증강 때문 . 작은 차이는 " 비슷하다 "
GPU 인데 느림	데이터를 .to(DEVICE) 로 안 옮겼거나 CPU 런타임 . 런타임 유형 확인
예측이 이상함	정규화 μ, σ 불일치 , 1 초 맞추기 누락 , model.eval() 누락

모양 (shape) 과 차원 (dim) 을 소리 내어 읽는 습관이 버그의 절반을 막는다 .

오늘 만든 것과 다음 단계

- ☐ Speech Commands 를 화자 기준으로 나누고 10 개 키워드를 골랐다
- ☐ 1 초 음성을 (1, 64, 101) Log-Mel 로 바꾸고 정규화했다
- ☐ 출력 크기와 파라미터 수를 공식으로 예측하고 확인했다
- ☐ 학습 루프를 직접 써서 KWSNet 을 학습했다
- ☐ 혼동 행렬로 헷갈리는 단어 쌍을 찾았다
- ☐ SpecAugment 와 predict_keyword() 로 미니 프로젝트를 완성했다

다음 : Day 6 · Whisper

단어 10 개 중 하나 고르기 → 어떤 문장이든 받아 적기 .

오늘의 Log-Mel · CNN · 학습 루프가 Whisper 인코더를 이해하는 바탕입니다 .

worksheet.html	40 문항
개념 워크시트 · 노션 제출	

animation.html	9 개 탭
인터랙티브 애니메이션	

D5_ 딥러닝오디오분류 _ 실습 .ipynb	빈칸 9
실습 노트북	

레포 위치 04-voice-ai/day05